

ASSIGNMENTS – OOADI

LECTURE 4

-
1. First we define our Bicycle class, seeing how this is the center of everything in this assignment.

```
package lecture4;

// We define our Bicycle class and include a few more or less random
// attributes. For the assignment we need only speed but we add a few more

public class Bicycle {
    private final String name; // Once set the name cannot be changed again!
    private double speedKmh;
    private final int gearCount;
    private final double weightKg;
    private final int order; // We used this for PriorityQueue to preserve insertion order

    // Constructor
    public Bicycle(String name, double speedKmh, int gearCount, double weightKg, int order) {
        this.name = name;
        this.speedKmh = speedKmh;
        this.gearCount = gearCount;
        this.weightKg = weightKg;
        this.order = order;
    }

    // Copy constructor
    public Bicycle(Bicycle other) {
        this.name = other.name;
        this.speedKmh = other.speedKmh;
        this.gearCount = other.gearCount;
        this.weightKg = other.weightKg;
        this.order = other.order;
    }

    // Convenient wrapper for the copy constructor
    public Bicycle copy() {
        return new Bicycle(this);
    }

    // We define a range of getters to access the Bicycle attributes
    public String getName() { return name; }
    public double getSpeedKmh() { return speedKmh; }
    public int getGearCount() { return gearCount; }
    public double getWeightKg() { return weightKg; }
    public int getOrder() { return order; }

    // We multiply speed by a factor (e.g., 0.75 or 1.25)
    public void adjustSpeed(double factor) {
        this.speedKmh *= factor;
    }
}
```

```
// We insert an override to provide a readable text description of our object
@Override
public String toString() {
    return String.format(
        "%s | speed: %.2f km/h | gears: %d | weight: %.1f kg",
        name, speedKmh, gearCount, weightKg
    );
}
}
```

Now, to create the required array, with 100 bicycles, we need to first initiate the array and then populate. One solution could be as follows:

```
Random rnd = new Random(42);

// Helper to format unique names: myBicycle_001 .. myBicycle_100
Function<Integer, String> nameGen = i -> String.format("myBicycle_%03d", i + 1);

Bicycle[] bikes = new Bicycle[100];

for (int i = 0; i < bikes.length; i++) {
    String name = nameGen.apply(i);
    // Random attributes
    double speed = 5.0 + rnd.nextDouble() * 35.0; // 5.0–40.0 km/h
    int gears = 1 + rnd.nextInt(21); // 1–21
    double weight = 6.0 + rnd.nextDouble() * 14.0; // 6.0–20.0 kg

    bikes[i] = new Bicycle(name, speed, gears, weight, i); // order=i
}
```

-
2. a) First, we create a stack to handle the operation, push all bicycles, and use a list to handle everything once we pop. The object order is reversed after this.

```
// Add all bicycles to a stack and decrease speed by 25% as they are removed
Stack<Bicycle> stack = new Stack<>();
for (Bicycle b : bikes) {
    stack.push(b.copy()); // use copies so other structures are unaffected
}

// Add all bicycles to a list once pop'ed
List<Bicycle> afterStack = new ArrayList<>();
while (!stack.isEmpty()) {
    Bicycle b = stack.pop();
    b.adjustSpeed(0.75); // decrease by 25%
    afterStack.add(b);
}
```

- b) We create a PriorityQueue and place copies of the original Bicycle objects here and carry out an ordering while increasing speed this time.

```
// We add all Bicycles to a PriorityQueue and increase speed by 25% in insertion order
PriorityQueue<Bicycle> pq = new PriorityQueue<>(Comparator.comparingInt(Bicycle::getOrder));
```

```
for (Bicycle b : bikes) {
    pq.add(b.copy()); // Again we use copies
}
```

```
List<Bicycle> afterPQ = new ArrayList<>();
while (!pq.isEmpty()) {
    Bicycle b = pq.poll(); // Finds smallest order bicycle
    b.adjustSpeed(1.25); // increase by 25%
    afterPQ.add(b);
}
```

c) Using a LinkedList we finally remove all Bicycles with a speed less than 10 km/h.

```
// Using a LinkedList we now remove all bicycles with speed < 10 km/h
LinkedList<Bicycle> linked = new LinkedList<>();
for (Bicycle b : bikes) {
    linked.add(b.copy()); // Again we work on a copy
}

int before = linked.size();
linked.removeIf(b -> b.getSpeedKmh() < 10.0);
int after = linked.size();
```

```
3. // Finally we place the remaining (speed > 10 km/h) bicycles in a HashMap
// and introduce a Scanner loop to find information on a specific bicycle
Map<String, Bicycle> mapByName = new HashMap<>();
for (Bicycle b : linked) {
    mapByName.put(b.getName(), b);
}

System.out.println("=== HashMap lookup by name ===");
System.out.println("Type a bicycle name like myBicycle_001 (or type 'exit' to quit).");
System.out.println("Tip: Available example names from remaining set: ");
linked.stream().limit(10).forEach(b -> System.out.print(b.getName() + " "));
System.out.println("\n");

try (Scanner scanner = new Scanner(System.in)) {
    while (true) {
        System.out.print("Enter bicycle name (or 'exit'): ");
        String input = scanner.nextLine();
        if (input == null) continue;
        input = input.trim();
        if (input.equalsIgnoreCase("exit")) {
            System.out.println("Exiting - Hasta la vista, baby!");
            break;
        }

        Bicycle found = mapByName.get(input);
        if (found != null) {
            System.out.println("Found: " + found);
        } else {
            System.out.println("No bicycle found with that name.");
        }
    }
}
```
